Veritabanı Performans Optimizasyonu ve İzleme

Proje - 1

Semih Cengiz - 20290233

BLM4522 - Ağ Tabanlı Paralel Dağıtım Sistemleri

Giriş

Bu projede örnek veritabanı kullanarak PostgreSQL ile performans analizi ve optimizasyon çalışmaları yapılmıştır. Çalışmanın amacı; sorgu performansını izleme, mevcut indeks yapısını kontrol etme, uzun süren sorguları analiz etme gibi konular içermektedir. Ayrıca disk alanı kullanımı ve veri yöneticisi rolleri gibi başlıklar incelenmiştir.

Kullanılan Veritabanı

Bu çalışmada PostgreSQL için özel olarak hazırlanmış Pagila veritabanı kullanılmıştır. Pagila DVD kiralama sistemine benzer bir yapıya ve tablolar sahiptir. Örneğin customer, film, mental ,payment gibi tablolar içermektedir.

Başlangıç Durumunun Analizi

table_name	row_count
film	1000
actor	200
customer	599
inventory	4581
rental	16044
payment	16044

Tabloların Satır Sayısı

Yukarıda gösterilen tablo veritabanının başlangıçta içerdiği tabloların büyüklüğünü göstermektedir. En yoğun tabloların rental ve payment olduğu görülüyor.

schemaname	table_name	total_size	total_size_bytes
public	rental	1920 kB	1966080
public	film	752 kB	770048
public	payment_p2007_03	512 kB	524288
public	film_actor	496 kB	507904
public	inventory	464 kB	475136
public	payment_p2007_04	440 kB	450560
public	payment_p2007_02	408 kB	417792

En Çok Yer Kaplayan Tablolar

Yukarıda tabloda tabloların diskte ne kadar yer kapladığı görülmektedir. Burada en fazla yer kaplayan rental tablosu olduğundan performans testlerinde özellikle kiralama işlemleriyle ilgili tabloların dikkate alınması gerektiğini göstermektedir. Ayrıca payment tablolarının parçalanmış halde bulunduğu görülmektedir.

schemaname	tablename	indexname	indexdef
public	actor	actor_pkey_incl	CREATE UNIQUE INDEX actor_pkey_incl ON public.actor USING btree (actor_id) INCLUDE (first_name, last_name)
public	actor	idx_actor_last_name	CREATE INDEX idx_actor_last_name ON public.actor USING btree (last_name)
public	film	film_fulltext_idx	CREATE INDEX film_fulltext_idx ON public.film USING gist (fulltext)
public	film	film_pkey	CREATE UNIQUE INDEX film_pkey ON public.film USING btree (film_id)
public	film	idx_fk_language_id	CREATE INDEX idx_fk_language_id ON public.film USING btree (language_id)
public	film	idx_fk_original_language_id	CREATE INDEX idx_fk_original_language_id ON public.film USING btree (original_language_id)
public	film	idx_title	CREATE INDEX idx_title ON public.film USING btree (title)
public	customer	customer_pkey	CREATE UNIQUE INDEX customer_pkey ON public.customer USING btree (customer_id)
public	customer	idx_fk_address_id	CREATE INDEX idx_fk_address_id ON public.customer USING btree (address_id)
public	customer	idx_fk_store_id	CREATE INDEX idx_fk_store_id ON public.customer USING btree (store_id)
public	customer	idx_last_name	CREATE INDEX idx_last_name ON public.customer USING btree (last_name)

Mevcut İndekslerin İncelenmesi

Başlangıç aşamasında zaten birçok indeksin tanımlı olduğu görülmektedir. Özellikle primary key ve foreign key alanlarında indeksler bulunmaktadır. Ayrıca film.title, actor.last_name ve customer.last_name gibi sorgularda sık kullanılabilecek alanlar için indeks tanımlamaları mevcuttur. Bu nedenle öncelikle optimizasyon için doğrudan indeksleri silmek yerine, önce hangi indekslerin gerçekten etkili olduğunun ölçülmesi planlanmaktadır.

Veritabanı izleme

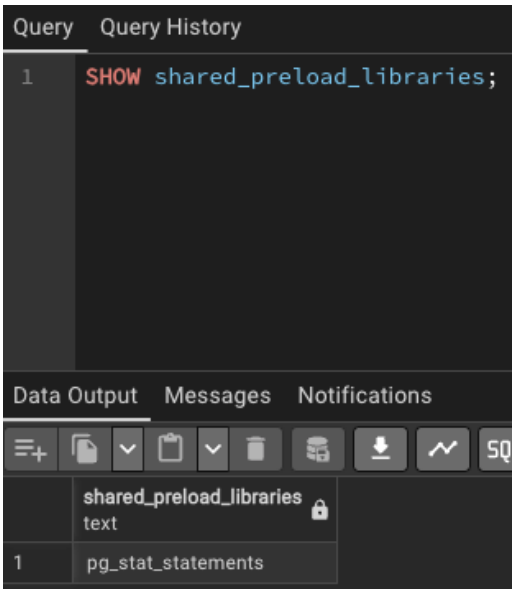
Bu bölümde pagila veritabanı için PostgreSQL'de bulunan **pg_stat_activity**, **pg_stat_statements** ve **log ayarları** modülü incelenmiştir. Bu araçlar sayesinde aktif sorgular, uzun süren işlemler ve performans problemi oluşturabilecek sorgular gözlemlenebilmektedir.

Modül Ayarları

Modüllerin çalışabilmesi için config dosyasında bazı değişiklikler yapmak gerekiyordu. postgresql.conf içindeki:

shared_preload_libraries = 'pg_stat_statements',

satırı varsayılan olarak boştu, onu pg_stat_statements ile değiştirdim. Bu adım, sorguların istatistiksel olarak izlenebilmesi için gereklidir.



SHOW Sorgusu İle Teyit

Eklenti Oluşturma

Yapılandırma ayarları tamamlandıktan sonra Pagila veritabanısında **pg_stat_statements** eklentisi oluşturulması gerekmektedir. Bu eklenti sayesinde veritabanısında çalışan sorguların çağrı sayısı, toplam çalışma süresi ve ortalama çalışma süresi gibi istatistikler elde edilebilmektedir.

```
CREATE EXTENSION IF NOT EXISTS pg_stat_statements;
SELECT * FROM pg_stat_statements LIMIT 5;
```

Sorgular

calls bigint	total_exec_time double precision	min_exec_time double precision	max_exec_time double precision	mean_exec_time double precision	stddev_exec_time double precision
5	0.44625	0.052375	0.138041	0.08925	0.035559982975249024
7	0.15487499999999998	0.011875	0.0375	0.022125000000000002	0.008901610127226582
7	6.663416	0.479125	1.4835	0.9519165714285714	0.4047192750236558
2	0.03229	0.007541	0.024749	0.016145	0.008604
5	0.029042	0.002666	0.00725	0.0058084	0.0016268111875691047

pg_stat_statements Sorgusundan Bir Parça

Yukarıdaki ekran görüntüsünde sorguların kaç defa çalıştığı, çalışma süreleri vs. görülebilmektedir.

Aktif Sorguları İzleme

PostgreSQL'de aktif oturumları ve çalışan sorguları izlemek için **pg_stat_activity** görünümü kullanılmıştır. Bu görünüm, o anda çalışan işlemleri, kullanıcı bilgisini, sorgu başlama zamanını ve bekleme durumlarını göstermektedir. Test amacıyla bir oturumda SELECT pg_sleep(20); sorgusu çalıştırılmış ve diğer oturumda **pg_stat_activity** ile bu sorgunun aktif olarak sistemde göründüğü gösterilmiştir. Böylece anlık sorgu izleme işlemi gerçekleştirilmiştir.

Query Query History

```
    usename,
    datname,
    state,
    wait_event_type,
    wait_event,
    now() - query_start AS duration,
    query
FROM pg_stat_activity
WHERE state <> 'idle'
ORDER BY duration DESC;
```

Data Output Messages Notifications

Showing rows: 1 to 2 Page No: 1 of 1

	pid integer	usename name	datname name	state text	wait_event_type text	wait_event text	duration interval	query text
1	27401	macbook	pagila	active	Timeout	PgSleep	00:00:08.6264...	SELECT pg_sleep(20);
2	27388	macbook	pagila	active	[null]	[null]	00:00:00	SELECT pid, usename, datname, stat

Aktif Sorguları Görme

Yukarıdaki tabloda diğer oturumda aktif olarak çalışan sleep sorgusunu, ne kadar zamandır çalıştığını vs. görebiliyoruz.

En Çok Zaman Harcayan Sorguları Tespit Etme

```
SELECT
    calls,
    round(total_exec_time::numeric, 2) AS total_ms,
    round(mean_exec_time::numeric, 2) AS mean_ms,
    rows,
    query
FROM pg_stat_statements
ORDER BY mean_exec_time DESC
LIMIT 10;
```

Ortalama Süreye Göre Sıralama

pg_stat_statements, sorguların çağrı sayısını, toplam planlama ve yürütme süresini, ortalama süreyi ve satır sayılarını gösterir.

	calls bigint	total_ms numeric	mean_ms numeric	rows bigint	query text
1	3	50003.45	16667.82	3	SELECT pg_sleep($1)
2	1	22.05	22.05	0	CREATE EXTENSION IF NOT E
3	3	40.68	13.56	2874	SELECT f.title, COUNT(r.r
4	3	36.17	12.06	1797	SELECT c.customer_id, c.
5	1	2.51	2.51	0	ALTER SYSTEM SET log_min_
6	2	4.44	2.22	0	ALTER SYSTEM SET deadlock
7	1	1.50	1.50	0	ALTER SYSTEM SET log_lock_
8	5	7.24	1.45	5	SELECT set_config($1,$2,$3)
9	19	18.23	0.96	442	SELECT DISTINCT att.attname
10	1	0.87	0.87	1	SELECT set_config($1,$2,$3)

Sorguların Çalışma Zamanına Göre Sıralanmış Hali

Yavaş Sorguları Loglama

Yavaş süren sorguları kayıt altına almak için **log_min_duration_statement** parametresi kullanılmıştır. Test amaçlı bu değer 0 olarak belirlenmiştir ki tüm sorguları görebilelim fakat isteğe bağlı bu değer değiştirilerek beklenenden fazla uzun süren sorgular loglanabilir, tespiti halinde sorunun nerede olduğu görülebilir.

```
2026-04-05 15:01:26.441 +03 [28542] LOG:  starting PostgreSQL 17.6 (Homebrew) on aarch64-apple-darwin24.4.0, compiled by Apple
clang version 17.0.0 (clang-1700.0.13.3), 64-bit
2026-04-05 15:01:26.442 +03 [28542] LOG:  listening on IPv6 address "::1", port 5432
2026-04-05 15:01:26.442 +03 [28542] LOG:  listening on IPv4 address "127.0.0.1", port 5432
2026-04-05 15:01:26.442 +03 [28542] LOG:  listening on Unix socket "/tmp/.s.PGSQL.5432"
2026-04-05 15:01:26.445 +03 [28546] LOG:  database system was shut down at 2026-04-05 15:01:25 +03
2026-04-05 15:01:26.449 +03 [28542] LOG:  database system is ready to accept connections
2026-04-05 15:01:33.620 +03 [28574] LOG:  duration: 4.047 ms  statement: SET DateStyle=ISO; SET client_min_messages=notice;
SELECT set_config('bytea_output','hex',false) FROM pg_show_all_settings() WHERE name = 'bytea_output'; SET
client_encoding='utf-8';
2026-04-05 15:01:33.625 +03 [28574] LOG:  duration: 0.296 ms  statement: SELECT version()
2026-04-05 15:01:33.627 +03 [28574] LOG:  duration: 2.535 ms  statement:
    SELECT
        db.oid as did, db.datname, db.datallowconn,
        pg_encoding_to_char(db.encoding) AS serverencoding,
        has_database_privilege(db.oid, 'CREATE') as cancreate,
        datistemplate
```

Log Dosyasının Bir Kısmı

Yukarıda PostgreSQL servisinin başlatıldığını, Sunucunun bağlantıyı kabul etmeye hazılandığını, her bir işlemin ne kadar sürdüğü görülebilmektedir. Burada minimum Log zamanı 0 yerine başka bir değer ayarlanırsa, filtreleme ile istenilen sonuç daha rahat görülebilir.

İndeks Yönetimi

Bu bölümde pagila veritabanı üzeride inde işlemlerini inceleyeceğiz. Amaç, sorgu performansını artırmak için uygun indeksleri belirlemek, mevcut indeksleri görüntülemek ve yapılan indeksleme işleminin sorgu sürelerine etkisini ölçmektir. Bunları yapabilmek için **pg_indexes**, **pg_stat_user_indexes** ve **EXPLAIN ANALYZE** araçlarını kullanacağım.

Mevcut İndeksleri Görüntüleme

Burada mevcut olan indeksler pg_indexes görünümü ile incelenmiştir. Aşağıdaki ekran fotoğrafında tablolar üzerinde hangi indekslerin tanımlı olduğu ve bu indekslerin hangi sütunlar için oluşturulduğu görülebiliyor.

schemaname	tablename	indexname	indexdef
public	actor	actor_pkey_incl	CREATE UNIQUE INDEX actor_pkey_incl ON public.actor USING btree (actor_id) INCLUDE (first_name, last_name)
public	actor	idx_actor_last_name	CREATE INDEX idx_actor_last_name ON public.actor USING btree (last_name)
public	address	address_pkey	CREATE UNIQUE INDEX address_pkey ON public.address USING btree (address_id)
public	address	idx_fk_city_id	CREATE INDEX idx_fk_city_id ON public.address USING btree (city_id)
public	category	category_pkey	CREATE UNIQUE INDEX category_pkey ON public.category USING btree (category_id)
public	city	city_pkey	CREATE UNIQUE INDEX city_pkey ON public.city USING btree (city_id)
public	city	idx_fk_country_id	CREATE INDEX idx_fk_country_id ON public.city USING btree (country_id)
public	country	country_pkey	CREATE UNIQUE INDEX country_pkey ON public.country USING btree (country_id)
public	customer	customer_pkey	CREATE UNIQUE INDEX customer_pkey ON public.customer USING btree (customer_id)
public	customer	idx_fk_address_id	CREATE INDEX idx_fk_address_id ON public.customer USING btree (address_id)

Şu Anki İndekslerin Bir Kısmı

İndeks Kullanım İstatistikleri

Burada hangi indeksin gerçekten kullanıldığını göreceğiz. **pg_stat_user_indexes** görünümünde yer alan değerler, şu ana kadar çalıştırılan sorgularda kullanılan birikmiş indeks kullanım istatistiklerini gösterir.

```
SELECT
    relname AS table_name,
    indexrelname AS index_name,
    idx_scan,
    idx_tup_read,
    idx_tup_fetch
FROM pg_stat_user_indexes
ORDER BY idx_scan DESC, relname;
```

	table_name name	index_name name	idx_scan bigint	idx_tup_read bigint	idx_tup_fetch bigint
1	customer	customer_pkey	24	24	24
2	film	film_pkey	6	6	6
3	inventory	inventory_pkey	6	6	0
4	rental	idx_fk_inventory_id	6	6	0
5	actor	idx_actor_last_name	0	0	0
6	actor	actor_pkey_incl	0	0	0
7	address	idx_fk_city_id	0	0	0

Kullanılan ve Kullanılmayan İndeksler

Tabloda görülen bilgiler şu ana kadar kadarki sorgulardan elde edilmiştir. Burada görülen satırla şunları ifade etmektedir.

table_name: indeksin ait olduğu tablo
index_name: indeksin adı
idx_scan: o indeks kaç kez taranmış
idx_tup_read: indeks üzerinden kaç kayıt okunmuş
idx_tup_fetch: tablodan gerçekten kaç satır çekilmiş

Fakat burada bazı indexlerin 0 görünmesi, bu indexin gereksiz olduğu anlamına gelmez. Veritabanı kullanıldıkça buradaki sayılar artacaktır ve az sayıda kalan indekslerin belki gereksiz olabileceğine karar verilebilir.

İndeks Eklenecek Sorgu

```
EXPLAIN ANALYZE
SELECT *
FROM payment
WHERE customer_id = 1;
```

EXPLAIN ANALYZE sorgusu

İndeks etkisini ölçebilmek için önce payment tablosu üzerinde filtreleme yapan bir sorgu girilmiştir. Bu sayede mevcut durumda sorgunun tabloyu nasıl taradığı gözlemlenecektir.

QUERY PLAN
Append (cost=0.28..67.09 rows=30 width=26) (actual time=0.367..2.827 rows=32 loops=1)
-> Index Scan using idx_fk_payment_p2007_01_customer_id on payment_p2007_01 payment_1 (cost=0.28..8.31 rows=2 width=26) (actual time=0.366..0.367 rows=2
Index Cond: (customer_id = 1)
-> Index Scan using idx_fk_payment_p2007_02_customer_id on payment_p2007_02 payment_2 (cost=0.28..8.37 rows=5 width=26) (actual time=0.387..0.389 rows=5
-> Seq Scan on payment_p0000_default payment_8 (cost=0.00..12.65 rows=3 width=26) (actual time=0.135..0.538 rows=3 loops=1)
Filter: (customer_id = 1)
Rows Removed by Filter: 609
Planning Time: 11.333 ms
Execution Time: 3.128 ms

EXPLAIN ANALYZE Çıktısı

Burada tabloda gerekli yerlerde Index Scan gerekli yerlerde Seq Scan kullanıldığı görülmüştür. Toplam çalışma süresi ise 3.128 ms'dir.

Index Eklendikten Sonra Performans

Aynı sorguyu bir de indeks ekleyip deneyeceğiz. **customer_id** sütunu üzerinde bir indeks oluşturulmuştu. Bu sayede toplam çalışma süresinin ne ölçüde değiştiğine bakacağız

```
CREATE INDEX idx_payment_customer_id
ON payment (customer_id);
```

İndeks Ekleme Komutu

QUERY PLAN
Append (cost=0.28..62.77 rows=30 width=26) (actual time=0.010..0.126 rows=32 loops=1)
-> Index Scan using idx_fk_payment_p2007_01_customer_id on payment_p2007_01 payment_1 (cost=0.28..8.31 rows=2 width=26) (actual time=0.010..0.010 rows=2
Index Cond: (customer_id = 1)
-> Index Scan using idx_fk_payment_p2007_02_customer_id on payment_p2007_02 payment_2 (cost=0.28..8.37 rows=5 width=26) (actual time=0.007..0.008 rows=5
Index Cond: (customer_id = 1)
-> Index Scan using idx_fk_payment_p2007_03_customer_id on payment_p2007_03 payment_3 (cost=0.28..8.39 rows=6 width=26) (actual time=0.006..0.008 rows=9
-> Index Scan using payment_p0000_default_customer_id_idx on payment_p0000_default payment_8 (cost=0.28..8.33 rows=3 width=26) (actual time=0.026..0.027 r
Index Cond: (customer_id = 1)
Planning Time: 2.077 ms
Execution Time: 0.175 ms

Yukarıda da görüldüğü gibi aynı sorgu çalıştırıldığında toplam çalışma süresi 0.175 ms'ye düşmüştür.

Ancak burada dikkat edilmesi gereken konu, her ne kadar çalışma süresi düşsede inden kullanımı fazladan diskte yer kaplayacaktır. Ayrıca çok düşük ve sıfır kullanım gösteren indexler kaldırılabilir fakat bu işlem yapılmadan önce indeksin uygulama açısından gerekli olup olmadığı analiz edilmelidir.

Sorgu İyileştirme

Bu bölümde sorgu iyileştirme işlemleri gerçekleştirilecektir. Amaç, uzun süren veya verimsiz çalışan bir sorguyu analiz etmek, neden yavaş çalıştığını incelemek ve aynı sonucu daha verimli üreten yeni bir sorgu yazmaktır. EXPLAIN ANALYZE ile bu işlemler gerçekleştirilecektir.

Sorgu Farkları

Burada aynı amaca hizmet eden iki farklı sorgu yazacağız ve ne kadar zamanda tamamlandıklarını analiz edeceğiz. Neden çalışma zamanlarının farklı olduğu tespit edilecektir.

```sql
EXPLAIN ANALYZE
SELECT
    c.customer_id,
    c.first_name,
    c.last_name,
    (
        SELECT SUM(p.amount)
        FROM payment p
        WHERE p.customer_id = c.customer_id
    ) AS total_payment
FROM customer c;
```

1.Sorgu

25	Planning Time: 0.498 ms
26	Execution Time: 22.640 ms

Çalışma Süresi

İlk aşamada müşteri bazlı toplam ödeme bilgisini getiren bir sorgu seçilmiştir. Bu sorguda her müşteri için ödeme toplamı ayrı bir alt sorgu ile hesaplanmaktadır. yapılan incelemede sorgunun çalışma zamanı incelenmiştir. Bu yapı, aynı işlemin çok sayıda tekrar yapılmasına neden olabildiği için sorgu performansı açısından incelenmeye uygundur.

```sql
EXPLAIN ANALYZE
SELECT
    c.customer_id,
    c.first_name,
    c.last_name,
    SUM(p.amount) AS total_payment
FROM customer c
LEFT JOIN payment p
    ON p.customer_id = c.customer_id
GROUP BY c.customer_id, c.first_name, c.last_name;
```

2.Sorgu

18	Planning Time: 0.299 ms
19	Execution Time: 9.689 ms

Çalışma Süresi

Aynı sonucu daha verimli üretmek amacıyla sorgu yeniden düzenlenmiştir. İyileştirilmiş sürümde her müşteri için ayrı alt sorgu çalıştırmak yerine `payment` tablosu `LEFT JOIN` ile birleştirilmiş ve toplam ödeme değeri `SUM` ile tek sorgu içinde hesaplanmıştır. Böylece sorgunun daha az tekrar içeren ve daha bütüncül bir planla çalışması hedeflenmiştir.

Veri Yönetici Rolleri

Bu bölümde amaç, farklı roller için erişim yönetimini göstermektir. PostgreSQL'de bunun temeli **role** kavramıdır; bir role hem kullanıcı gibi davranabilir hem de grup gibi yetki taşıyabilir. Roller veritabanı nesnelerine sahip olabilir ve bu nesneler üzerinde başka rollere yetki verebilir. Yetkiler `GRANT` ve `REVOKE` ile yönetilir; roller ayrıca grup mantığıyla başka rollere üye de yapılabilir.

```sql
CREATE ROLE pagila_admin LOGIN PASSWORD 'inanilmazguclusifre123!admin';
CREATE ROLE pagila_readonly LOGIN PASSWORD 'inanilmazguclusifre123!readonly';
CREATE ROLE pagila_dataentry LOGIN PASSWORD 'inanilmazguclusifre123!dataentry';
```

3 Farklı Rol

Farklı erişim seviyelerinde 3 tane rol oluşturdum. **pagila_admin** yönetici rolü, **pagila_readonly** sadece okuma rolü ve **pagila_dataentry** veri giriş rolü olarak tanımlanmıştır. Bu roller sisteme kullanıcı gibi bağlanabilmektedir.

Erişim Yetkisi

Roller oluşturulduktan sonra ilgili kullanıcıların veritabanına bağlanabilmesi için `CONNECT` yetkisi verilmiştir. Ayrıca tabloların bulunduğu `public` şeması üzerinde `USAGE` yetkisi tanımlanmıştır.

```sql
GRANT CONNECT ON DATABASE pagila TO pagila_readonly, pagila_dataentry, pagila_admin;
GRANT USAGE ON SCHEMA public TO pagila_readonly, pagila_dataentry, pagila_admin;
```

Erişim Yetkisi

Tablo Bazında Yetki

pagila_readonly rolüne **customer** ve **payment** tabloları üzerinde yalnızca **SELECT** yetkisi verilmiştir. **pagila_dataentry** rolüne bu tablolar üzerinde **SELECT**, **INSERT** ve **UPDATE** yetkileri tanımlanmıştır. **pagila_admin** rolüne ise aynı tablolar üzerinde tüm yetkiler verilmiştir. Bu yapı sayesinde kullanıcıların yalnızca ihtiyaç duydukları işlemleri yapabilmesi sağlanmıştır.

```sql
GRANT SELECT ON customer TO pagila_readonly;
GRANT SELECT ON payment TO pagila_readonly;

GRANT SELECT, INSERT, UPDATE ON customer TO pagila_dataentry;
GRANT SELECT, INSERT, UPDATE ON payment TO pagila_dataentry;

GRANT ALL PRIVILEGES ON customer TO pagila_admin;
GRANT ALL PRIVILEGES ON payment TO pagila_admin;
```

Verilen Yetkileri Listeleme

Verilen yetkiler, **information_schema.role_table_grants** görünümü kullanılarak görülebilmektedir. Bu şekilde hangi role, hangi tablo üzerinde hangi yetkinin verildiği açık biçimde gözlemlenebilir.

```
SELECT
    grantee,
    table_schema,
    table_name,
    privilege_type
FROM information_schema.role_table_grants
WHERE table_name IN ('customer', 'payment')
ORDER BY grantee, table_name, privilege_type;
```

Yetki Listesini Görme Sorgusu

	grantee name	table_schema name	table_name name	privilege_type character varying
1	pagila_admin	public	customer	DELETE
2	pagila_admin	public	customer	INSERT
3	pagila_admin	public	customer	REFERENCES
4	pagila_admin	public	customer	SELECT
5	pagila_admin	public	customer	TRIGGER
18	pagila_dataentry	public	payment	INSERT
19	pagila_dataentry	public	payment	SELECT
20	pagila_dataentry	public	payment	UPDATE
21	pagila_readonly	public	customer	SELECT
22	pagila_readonly	public	payment	SELECT

Verilen Yetkiler

Verilen Yetkilerin Geri Alınması

Yalnızca yetki verme değil, aynı zamanda yetki alma işlemleri de yapılabilmektedir. Örnek olarak **pagila_dataentry** rolünün **payment** tablosu üzerindeki **UPDATE** yetkisi geri alınmıştır. **GRANT** ve **REVOKE** komutları ile yetkiler düzenlenebilmektedir.

```
REVOKE UPDATE ON payment FROM pagila_dataentry;
```

Yetki Alma